

# Tutorial - Semana 6, Encontro 1

---

## Introdução

---

A Semana 5 encerrou com o contrato `Interactable` e Signals sustentando pelo menos um objeto interativo funcional (Door) e um segundo objeto próprio de cada grupo (por exemplo, Lever), reagindo através do `InteractionComponent` do Player. Esta semana muda de problema: até agora, qualquer dado de um item (nome, valor, ícone) estaria hardcoded dentro de um script de comportamento, misturando dado de design com lógica de gameplay. Este encontro resolve isso com o Resource customizado `ItemData` — uma estrutura de dados independente de qualquer Scene, reutilizável por qualquer objeto que precise descrever um item.

## Objetivos da semana

---

- Explicar Resource customizado como estrutura de dados desacoplada da lógica de gameplay.
- Diferenciar dado de design (Resource) de comportamento de gameplay (Node/script).
- Criar um Resource customizado ( `ItemData` ) para itens do Vertical Slice.

## Resultado esperado ao final da semana

---

Ao final da Semana 6 (Encontros 1 e 2), cada grupo terá uma classe `ItemData` (Resource customizado) com pelo menos um Enum de categoria, aplicada a um conjunto próprio de itens coletáveis. Este tutorial cobre apenas o **Encontro 1**: a criação da classe `ItemData` e de pelo menos duas instâncias `.tres` de itens distintos.

## Pré-requisitos

---

- Contrato `Interactable`, Signals e pelo menos dois objetos interativos (Door e o objeto próprio do grupo) da Semana 5, já testados.
-

# Antes de começar

---

## O que o estudante deverá possuir antes desta semana

---

- O projeto herdado da Semana 5, com `Door.tscn` e o segundo objeto interativo do grupo funcionando através do `InteractionComponent` do Player, sem alterações pendentes.

## Arquivos necessários

---

- Nenhum arquivo externo adicional.

## Assets utilizados

---

- Ícones simples (texturas 2D) para representar itens no Inspector, se disponíveis no pacote de assets do projeto; opcionais para este encontro.

## Projeto esperado

---

- Projeto aberto no Godot 4.7, com o `Interactable` e os Signals da Semana 5 já testados.
- Pasta `resources/items/` criada (ou a ser criada neste encontro) conforme a estrutura do `PROJECT_ARCHITECTURE.md` (seção 8).

### Imagem sugerida

Objetivo: mostrar o script de um Resource customizado no editor de script do Godot, com `class_name ItemData` e campos `@export` declarados. Enquadramento: captura de tela do editor de script (Script Editor) com o arquivo `item_data.gd` aberto. Elementos importantes: a linha `extends Resource`, a linha `class_name ItemData`, ao menos dois campos `@export`. Destaque: a palavra-chave `extends Resource`, que diferencia este script de um script de Node comum. Legenda sugerida: "Script ItemData estendendo Resource, com campos exportados para nome, ícone e valor."

---

# Parte 1 — O problema: dado de design misturado à lógica

---

## Objetivo

---

Entender por que armazenar nome, ícone e valor de um item dentro do script de comportamento de um objeto (por exemplo, dentro de `chest.gd`) cria um problema de manutenção.

## Conceito

---

Toda engine de jogos com conteúdo além do mínimo precisa resolver o mesmo problema: como permitir que quem projeta o conteúdo (itens, inimigos, fases) trabalhe sem editar código a cada novo item. Se cada item coletável exigisse uma classe própria ou valores hardcoded no script do objeto que o representa, adicionar um item novo significaria mexer em lógica, não em dado — e duplicar variáveis como `nome`, `valor` e `icone` em cada Scene que usa um item.

O Resource customizado resolve isso definindo uma estrutura de dados serializável, independente de qualquer Scene ou Node específico. O mesmo `ItemData` pode ser reutilizado por um baú, uma prateleira de loja ou um slot de inventário — todos lendo os mesmos campos sem duplicar a definição. Esse desacoplamento entre dado e lógica é o mesmo princípio, em escala menor, que já apareceu no contrato `Interactable` da Semana 5: ali se desacoplou comportamento de quem o chama; aqui se desacopla dado de quem o consome.

## Passo a passo

---

Esta parte não possui etapas de implementação no editor — é a base conceitual antes da Parte 2.

1. Revisar com a turma um exemplo hipotético: um `Chest.tscn` com `nome`, `valor` e `icone` declarados como variáveis soltas dentro do próprio script do baú.
2. Perguntar: "se este mesmo item também precisasse aparecer em uma prateleira de loja ou em um slot de inventário, essas três variáveis seriam duplicadas em cada lugar?"
3. Concluir que a resposta correta é não — o dado deve existir uma única vez, em uma estrutura própria, consultada por qualquer Scene que precise dele.

## Resultado esperado

---

A turma entende a diferença entre "dado de design" (o que é um item) e "lógica de gameplay" (o que acontece quando o item é coletado), e por que o primeiro não deve viver dentro do script do segundo.

## Verificando

---

1. Confirmar que os estudantes conseguem citar, em suas próprias palavras, um exemplo de dado que se repetiria se não existisse o Resource.

## Problemas comuns

---

- Confundir "dado de design" com "configuração fixa que nunca muda": reforçar que mesmo dados fixos (nome de um item) se beneficiam de viver fora do script quando reutilizados por múltiplas Scenes.

## Boas práticas

---

- Manter esta discussão breve, reservando a maior parte do encontro para a implementação nas Partes 2 e 3.

## Comparação com Unity

---

A Unity resolve o mesmo problema com `ScriptableObject` — um equivalente quase direto do Resource do Godot: ambos são estruturas de dados serializáveis, editáveis pelo Inspector, independentes de instância em cena. A diferença arquitetural relevante é sutil: no Godot, Resource é a mesma classe-base usada internamente por texturas, cenas e scripts, então customizar um Resource é estender um mecanismo já onipresente no motor; na Unity, `ScriptableObject` é uma classe-base separada, criada especificamente para esse padrão de dado-como-asset. O conceito universal — dado de design como asset independente de lógica — é idêntico nas duas engines.

---

## Parte 2 — Criando o Resource `ItemData`

---

### Objetivo

---

Criar a classe `ItemData`, estendendo `Resource`, com campos exportados básicos, e gerar pelo menos duas instâncias `.tres` de itens distintos.

## Conceito

Um Resource customizado no Godot é um script comum que estende a classe `Resource` em vez de estender um tipo de Node. Ao declarar `class_name`, esse script passa a aparecer como um novo tipo selecionável ao criar um recurso pelo FileSystem Dock. Campos marcados com `@export` ficam visíveis e editáveis no Inspector para cada instância `.tres` gerada a partir da classe — exatamente como aconteceria com um Node, mas sem que o dado esteja preso a nenhuma Scene.

Seguindo a convenção do `PROJECT_ARCHITECTURE.md` (seção 9), a classe usa PascalCase (`ItemData`) e cada instância `.tres` usa snake\_case descrevendo o item concreto (por exemplo, `item_moeda.tres`).

## Passo a passo

1. No FileSystem Dock, crie a pasta `resources/items/`, caso ainda não exista, seguindo a estrutura do `PROJECT_ARCHITECTURE.md` (seção 8).
2. Clique com o botão direito em `scripts/` (ou pasta equivalente do projeto) e crie um novo script: **Create New > Script**.
3. Nomeie o arquivo `item_data.gd`, salvando em `scripts/resources/` (crie a subpasta se não existir).
4. No script, defina a herança e o nome de classe:

```
extends Resource
class_name ItemData
```

5. Declare os campos exportados básicos do item:

```
@export var nome: String = ""
@export var icone: Texture2D
@export var valor: int = 0
@export var descricao: String = ""
```

6. Salve o script (**Ctrl+S**).
7. No FileSystem Dock, clique com o botão direito na pasta `resources/items/` e selecione **New Resource**.
8. Na janela de busca, digite `ItemData` e selecione a classe recém-criada.
9. Nomeie o arquivo gerado como `item_moeda.tres`.
10. Com o arquivo `.tres` selecionado, preencha os campos no Inspector (`nome`, `valor`, `descricao`; `icone` se houver textura disponível).

11. Repita os passos 7 a 10 para criar uma segunda instância, `item_chave.tres`, com valores distintos.

## Resultado esperado

---

A pasta `resources/items/` contém a classe `ItemData` e pelo menos duas instâncias `.tres` (`item_moeda.tres`, `item_chave.tres`), cada uma com valores próprios preenchidos no Inspector.

## Verificando

---

1. Selecione `item_moeda.tres` no FileSystem Dock e confirme, no Inspector, que os campos `nome`, `icone`, `valor` e `descricao` aparecem preenchidos.
2. Selecione `item_chave.tres` e confirme que os valores são diferentes dos de `item_moeda.tres`.
3. Abra `item_data.gd` e confirme que a classe estende `Resource` (não `Node` ou `Node3D`).

## Problemas comuns

---

- Colocar lógica de gameplay (por exemplo, um efeito ao coletar o item) dentro do próprio `ItemData`: reforçar que o `Resource` guarda apenas dado — a reação pertence à `Scene` que o consome (o `Chest` ou o `Pickup`, na Semana seguinte).
- Confundir a classe `ItemData` com uma instância `.tres`: reforçar que a classe define a estrutura e cada `.tres` é um dado concreto preenchido a partir dela.
- Esquecer de exportar ( `@export` ) os campos, tornando-os invisíveis no Inspector ao criar a instância: orientar verificação no Inspector antes de seguir para o próximo item.
- Estender `Node` em vez de `Resource` por hábito: um `Resource` não faz parte da árvore de `Scene` e não deve estender nenhum tipo de `Node`.

## Boas práticas

---

- Dar valores padrão ( `= ""` , `= 0` ) aos campos exportados, evitando instâncias com campos vazios por esquecimento.
- Nomear cada instância `.tres` descrevendo o item concreto ( `item_moeda.tres` ), nunca de forma genérica ( `resource1.tres` ), conforme a seção 9 do `PROJECT_ARCHITECTURE.md`.
- Manter todas as instâncias de item na mesma pasta ( `resources/items/` ), nunca soltas na raiz do projeto.

## Comparação com Unity

---

Na Unity, o mesmo resultado exigiria criar uma classe C# com `[CreateAssetMenu]` estendendo `ScriptableObject`, com campos públicos ou `[SerializeField]` equivalentes aos `@export` do Godot. Cada instância seria gerada pelo menu de criação de assets do Editor, do mesmo modo que o Godot gera um `.tres` pelo FileSystem Dock. A diferença prática é que a Unity exige o atributo `[CreateAssetMenu]` explicitamente para que o tipo apareça no menu de criação, enquanto no Godot qualquer Resource com `class_name` já fica disponível automaticamente na busca de **New Resource**.

---

## Ao final do encontro

---

Ao final do Encontro 1 da Semana 6, o projeto do Vertical Slice deve conter:

- O `GameManager` e o `SaveManager` da Semana 4, sem nenhuma alteração.
- O contrato `Interactable`, `Signals`, `Door` e o objeto próprio de cada grupo da Semana 5, sem nenhuma alteração.
- A classe `ItemData` (Resource customizado), com campos `nome`, `icone`, `valor` e `descricao`, salva em `scripts/resources/item_data.gd`.
- Pelo menos duas instâncias `.tres` de itens distintos (`item_moeda.tres`, `item_chave.tres`), salvas em `resources/items/`.

Segundo o `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 2), este resultado corresponde ao início do item "ItemData (Resource + Enum)" do roadmap, a ser completado no Encontro 2 com a adição do Enum de categoria.

## Desafio

---

Cada estudante adiciona ao próprio `ItemData` um campo não demonstrado neste encontro (por exemplo, `peso`, `raridade` ou uma segunda `descricao` curta para tooltip), mantendo a estrutura de Resource como ponto de entrada dos dados. O desafio reutiliza exatamente a mesma classe criada na Parte 2, sem exigir nenhum conteúdo do Encontro 2.

## Checklist

---

- ☐ Classe `ItemData` criada, estendendo `Resource`, com `class_name` definido

- ☐ Pelo menos quatro campos exportados ( `nome` , `icone` , `valor` , `descricao` ) visíveis no Inspector
- ☐ Pelo menos duas instâncias `.tres` criadas em `resources/items/` , com valores próprios preenchidos
- ☐ Campo adicional do desafio incluído no `ItemData` e testado no Inspector
- ☐ Nenhuma lógica de gameplay presente dentro do script `item_data.gd`

## Glossário

---

- **Resource:** classe-base do Godot para estruturas de dados serializáveis, independentes da árvore de Scene; usada internamente por texturas, cenas e scripts, e customizável para dados de design próprios do projeto.
- **Resource customizado:** um Resource com `class_name` próprio, definindo uma estrutura de dados específica do projeto (por exemplo, `ItemData` ).
- **.tres** : formato de arquivo de texto do Godot para uma instância salva de um Resource, análogo a um asset de dado editável pelo Inspector.
- **@export** : anotação do GDScript que expõe uma variável no Inspector, tornando-a editável por instância sem alterar o script.

## Referências

---

- Godot Documentation — Resources:  
<https://docs.godotengine.org/en/stable/tutorials/scripting/resources.html>
- Godot Documentation — GDScript ( `@export` ):  
[https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript\\_exports.html](https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript_exports.html)
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Unity Manual (consulta comparativa) — ScriptableObject:  
<https://docs.unity3d.com/Manual/class-ScriptableObject.html>
- Canais recomendados para consulta complementar (não substituem a documentação oficial):  
GDQuest (<https://www.youtube.com/@GDQuest>), Clear Code (<https://www.youtube.com/@ClearCode>)